Project Plan – MUA600

Name: Gabriel Danho

Date: 2026-05-20

1. Chosen approach

Hybrid: CMAS for the agent system and *Python* (*Ollama*) for the LLM planner.

CMAS is the natural fit for the agent system itself. It has native *Modbus TCP* support on every variable, a built-in negotiation and booking mechanism through *abstract interfaces*, and an explicit *Plug & Produce* model where agents can be discovered at runtime. This maps directly onto the R1–R4 requirements with little impedance mismatch. The Lab 2 crane example already provides a working Modbus connection and a model with Source, Process, Sink, Crane and Product agents, which is the right starting skeleton.

CMAS Structured Text is not a good host for an *LLM* client, so for R5 the LLM runs in a separate Python process using Ollama with llama3 (*Lab 5*). The Python sidecar produces *structured JSON* orders and writes them to a shared file; a *CMAS OrderIntake* agent polls the file and deploys Part agents accordingly. The *LLM* never touches *Modbus* directly (it only produces validated orders that the agent system consumes).

Contrasted with the AgentPy path (*Lab 4*): AgentPy would make the LLM integration slightly more direct, but the negotiation and Plug & Produce mechanisms would have to be coded from scratch, and the Modbus layer would need a custom pymodbus wrapper. *CMAS* gives those for *free*.

2. Agent decomposition

Resource agents (one instance each, location loaded from runtime config for R3):

- **Crane:** owns Modbus signals setX, setY, vacuum, posX, posY. Provides a single transport skill that accepts (fromX, fromY, toX, toY). Has *no knowledge* of part types, process plans, or station identities.
- **Process1, Process2:** each owns its own run, isRunning, sensor signals and its current X coordinate. Provides a process skill plus a *capability descriptor* (which operations it can perform) so Parts can discover alternatives for R4.
- **Source1, Source2:** owns its sensor and X coordinate and the responsibility of deploying a new Part agent when a part is generated (*deploybyname() pattern from the CMAS docs*).
- **Sink:** owns its X coordinate; final destination for finished parts and the sink agent exists not for its behaviour, but to *own its location data*.

Product agents (Each physical part in the simulation has one Part agent shadowing it):

- **Part:** owns its process plan (type 1: Process1 → Sink; type 2: Process2 → Process1 → Sink). The Part is the active agent: it requests transport from the Crane and processing from a Process via *abstract* interfaces, and re-plans on failure. The Part is the *only agent* that knows its full route through the factory.

R5 extension agent (*Coordination agents*):

- **OrderIntake:** *CMAS* agent that polls a *JSON* file produced by the *Python LLM* sidecar and deploys the requested Part agents. This is the *only* agent in our system that has any awareness

of *orders*. The *rest* of the agents *don't know* what an order is and they just see Part agents appearing at Sources.

3. Communication and discovery

All inter-agent communication uses *CMAS abstract interfaces* with negotiation, *not hardcoded* references.

- **Discovery:** each resource exposes a *typed interface* (e.g. ifTransport on the Crane, ifProcess on the Processes). A Part declares an *abstract interface* and calls agreeandbook(), which *negotiates and binds* to a compatible resource at runtime. This is the same mechanism shown in the CMAS documentation and the Lab 2 example.
- **Locations:** each resource agent *exposes* its own X coordinate as an *interface variable*. The Crane reads coordinates from whichever Source / Process / Sink interface it is currently bound to, rather than holding a coordinate table. This satisfies R3 directly.
- **Mutual exclusion:** an interface that is booked *cannot* be re-booked. Two Parts that both want Process1 will be serialized naturally by the booking mechanism (the second call to *agreeandbook()* blocks until the first Part calls *_unbook()*).
- **LLM bridge:** the Python sidecar writes orders.json; OrderIntake reads it. *No direct LLM-to-Modbus path*, satisfying the R5 requirement that the LLM does not write to Modbus directly.

4. Negotiation (targeting grade A)

CMAS's built-in negotiate() & agreeandbook() handles compatibility matching by *interface* name and *variable* signature. On top of this, a lightweight *bid mechanism* in the *onNegotiate function* decides between compatible providers for R4 (failure recovery).

- **CFP:** a Part needing a processing operation declares an abstract *ifProcess* interface and calls agreeandbook(). CMAS broadcasts the request to all compatible Process agents.
- **Propose:** each Process implements *onNegotiate(planning):* it returns *1 (accept)* if it is not currently failed and can perform the requested operation type, otherwise *-1 (reject)*. A bid metric (current queue length) is exposed as a *specialisation variable* so the Part can prefer the least-loaded Process when both accept.
- **Accept / Reject:** *CMAS selects* a binding from the accepting offers. The Part proceeds with the booked interface. On failure (R4), the *Part* calls *_unbook () and re-runs agreeandbook()*, the *failed Process* now returns *-1* and the *alternative* is selected *automatically*.

5. Extension – R5 (LLM planner)

Order-intake LLM: natural-language order → structured JSON → real production.

Architecture: A Python script using Ollama (llama3, already installed from Lab 5) takes *natural-language* input such as “make three type-1 parts and two type-2 parts”. The *script* prompts the *LLM* with a strict schema and asks for *JSON* output of the form {"orders": [{"type": 1, "count": 3}, {"type": 2, "count": 2}]}. The Python wrapper validates the JSON against the schema before writing

it to *orders.json*. Invalid output is rejected and re-prompted up to a small retry limit, then a clear error is logged. The *OrderIntake CMAS* agent polls *orders.json* and deploys the appropriate Part agents via *deploybyname()*.

Demonstration plan: open a console, type a *natural-language* order, show the *parsed JSON*, and watch the *simulation* produce exactly those parts in the way the plan specified with the rest of the agent system running unchanged from R4.

6. Target grade and requirements

Target: Grade A (R1 + R2 + R3 + R4 + R5).

Safe floor: Grade B (R1 + R2 + R3 + R4). The full agent system, Plug & Produce, and failure recovery are achievable within the project timeline using CMAS alone. R5 builds on the Lab 5 Ollama / AutoGen work and is the stretch.

Order of implementation: R1 (one product type, full skeleton) → R2 (second product type, no Crane change) → R3 (locations from a JSON config file, Crane fetches dynamically) → R4 (failure flag on Process, re-negotiation on the Part) → R5 (Python LLM sidecar + OrderIntake agent).

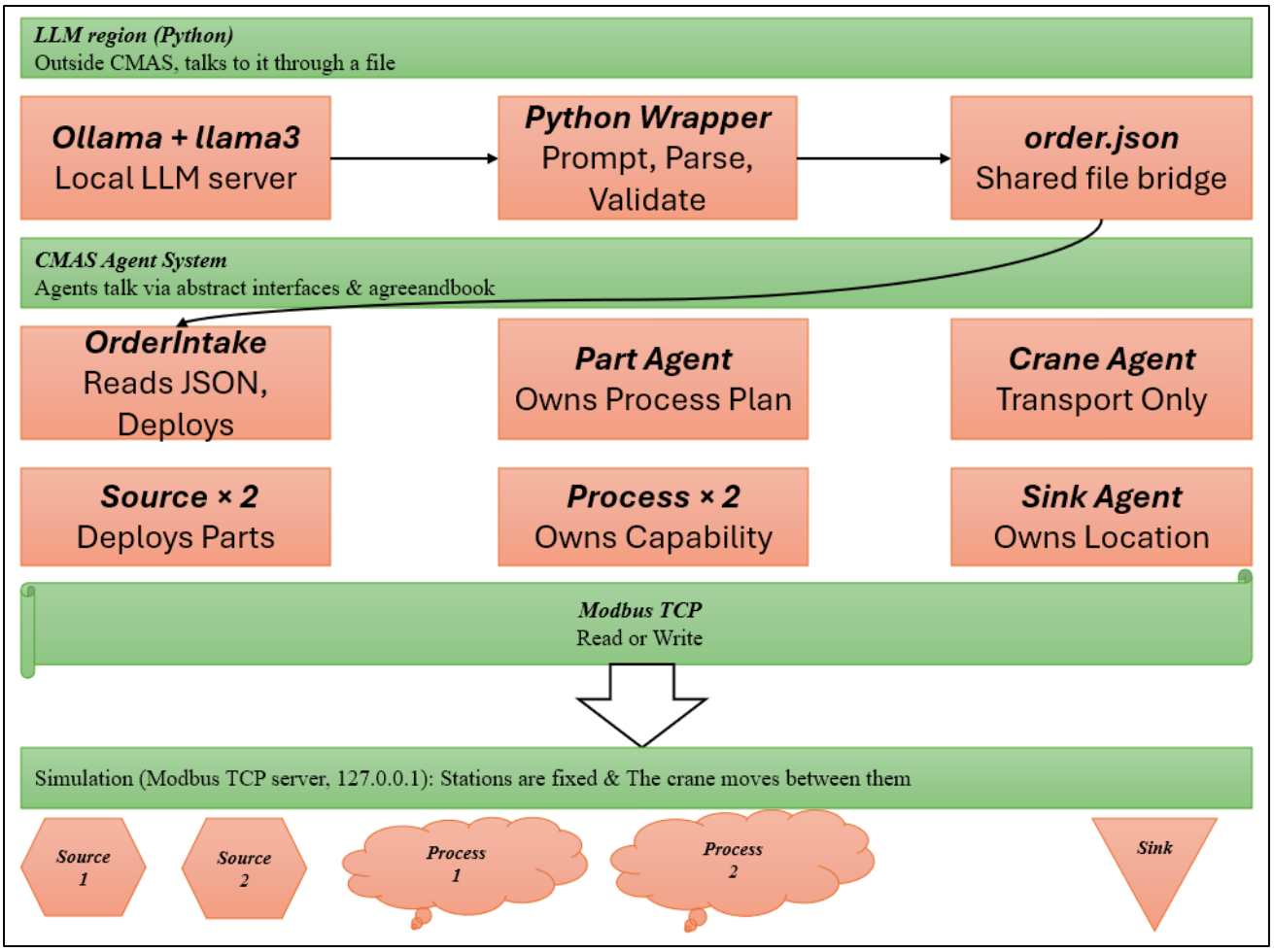
7. Risks and open questions

Risks I expect to be hard:

- **Collision (free Crane motion with concurrent Parts):** the Crane skill must serialize pick & place operations even while multiple Parts hold bookings on different Processes. *Plan*: keep the Crane's *transport skill* as the only critical section (bookings on Processes *do not imply* Crane access).
- **Timing of process completion:** the docs warn that *isRunning* must transition *back* to *0* before a process is considered *done*, and that a brief *delay* follows part generation before the sensor registers. *Plan*: explicit *polling* loops with *bounded* waits in each skill.
- **LLM JSON reliability:** llama3 is small and can produce malformed JSON. *Plan*: schema validation with retries, and a clear failure path that the demo can show on purpose.

Questions for the teacher:

- **Configuration source for R3:** is a *JSON file* in the project directory acceptable, or do you prefer command-line arguments or a CMAS prompt?
- **Failure trigger for R4:** would you prefer the failure to be triggered by a console keystroke, an *extra Modbus register*, or by killing a CMAS agent? Each is feasible (I want to pick the one that demos best).
- **R5 scope:** is order-intake (*natural language → JSON → parts*) sufficient for R5, or do you expect the LLM to also do plan validation & routing decisions on top?


LLM region (Python)
Outside CMAS, talks to it through a file
Ollama + llama3
Local LLM server
Python Wrapper
Prompt, Parse, Validate
order.json
Shared file bridge
CMAS Agent System
Agents talk via abstract interfaces & agreeandbook
OrderIntake
Reads JSON, Deploys
Part Agent
Owns Process Plan
Crane Agent
Transport Only
Source × 2
Deploys Parts
Process × 2
Owns Capability
Sink Agent
Owns Location
Modbus TCP
Read or Write
Simulation (Modbus TCP server, 127.0.0.1): Stations are fixed & The crane moves between them
Source 1
Source 2
Process 1
Process 2
Sink